Internal Training Program

Reading & Maintaining Our SOAP Services (ASP.NET Core + SoapCore + WCF Contracts)

Audience: New engineer, already comfortable with C#, OOP, SQL Server, ADO.NET, ASP.NET Core Web API, DI, middleware, WinForms, and basic HTML/CSS/JS. Currently learning Web Forms (our frontend stack).

Goal: Not to become a WCF expert. The goal is to be able to open any `.asmx`-style service in this codebase, understand what it's doing, trace a request from the Web Forms frontend down to SQL Server and back, and confidently make changes without breaking anything.

How to read this course: Every module follows the same shape — objectives, theory, why it exists, where you'll see it in enterprise code, how it maps to *our* codebase, examples, a diagram, best practices, common mistakes, a real scenario, exercises, a quiz, and a summary. Skim the theory, focus on the "how it relates to our company" sections — that's the part that will actually make you productive.

Module 1 — SOAP Fundamentals (Just Enough)

Learning Objectives

- Understand what SOAP actually is, at the wire level.
- Understand why a company would still use SOAP in 2026.
- Recognize a SOAP request/response when you see one.

Theory, explained simply

SOAP (Simple Object Access Protocol) is just a **contract for formatting messages as XML**, sent over HTTP (usually POST). That's it. There's no magic. A SOAP message is an XML envelope with a header and a body:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header></soap:Header>
  <soap:Body>
    <WebServiceRQ xmlns="http://tempuri.org/">
      <strRQ>&lt;Request&gt;&lt;User&gt;jdoe&lt;/User&gt;&lt;/Request&gt;</
strRQ>
    </WebServiceRQ>
  </soap:Body>
</soap:Envelope>
```

Compare that to REST/JSON, which you already know from ASP.NET Core Web API:

```
{ "strRQ": "<Request><User>jdoe</User></Request>" }
```

Same idea — "send some structured data to an endpoint, get structured data back" — just a different wrapper format. SOAP additionally standardizes things like: how faults/errors are represented, how

the service describes itself (WSDL — Module 7), and how headers can carry things like authentication tokens.

Why this concept exists

SOAP was created in the late 1990s/2000s when companies needed strict, machine-verifiable contracts between systems built in different languages (a Java system talking to a .NET system, for example). It predates REST/JSON becoming the default. Airlines, banks, insurance, healthcare, and government systems adopted SOAP heavily during that era — and because these integrations are expensive to replace, a lot of that SOAP infrastructure is still running today, unchanged, 20 years later.

Where it appears in enterprise applications

- Airline/GDS booking systems (this is extremely common — flight search, booking, ticketing APIs are very often still SOAP).
- Banking core systems and payment gateways.
- Government and healthcare interoperability (many mandated formats are SOAP-based).
- B2B integrations with large legacy partners who never migrated off SOAP.

How it relates to our company

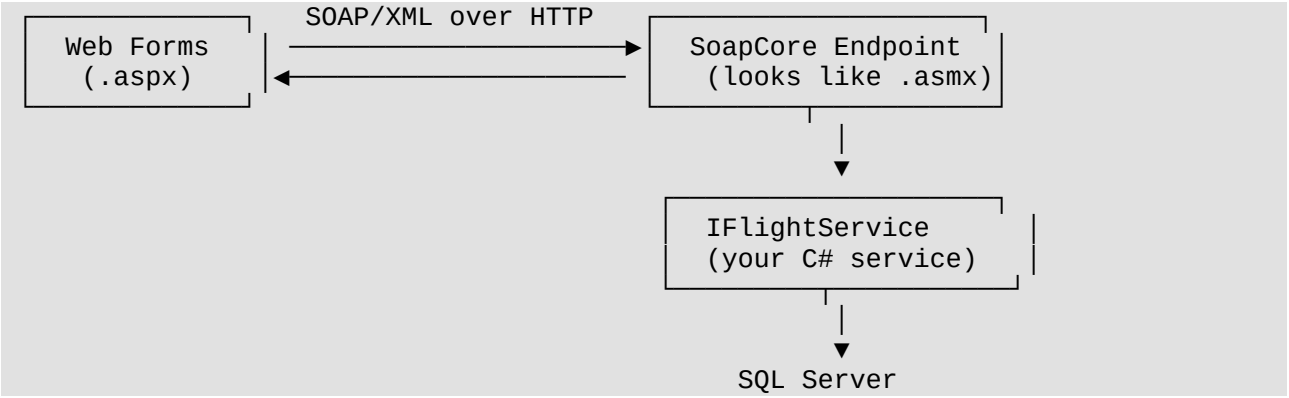
Your company's `IFlightService` is a strong signal this is a **travel/flight booking system** — this is exactly the kind of domain where SOAP survives because airline GDS/backend partners (Amadeus, Sabre, Travelport, etc.) still speak SOAP or SOAP-like XML. Your job is not to modernize this away — it's to read and extend it safely.

Practical example

Here's the mental model to hold onto for the rest of this course:

```
Web Forms Page (.aspx)
        |
        |  builds a SOAP request (XML string)
        v
   SOAP Endpoint (SoapCore, looks like .asmx)
        |
        v
  Controller-equivalent: your [ServiceContract] implementation
        |
        v
  Business logic parses XML manually (XmlDocument)
        |
        v
     SQL Server
        |
        v
  Business logic builds an XML response
        |
        v
  SoapCore wraps it back into a SOAP envelope
        |
        v
Web Forms Page reads the XML response
```

Diagram



Best practices

- Don't assume "SOAP" means "old/bad." It means "strict contract, XML payload." Judge the code on its own merits.
- Always look at an actual sample request/response before touching a service — SOAP code is much easier to understand with a real XML example in front of you.

Common mistakes

- Assuming SOAP = ASMX = classic ASP.NET. It isn't, in your case (see Module 8).
- Trying to "fix" the XML-string-based contract by changing method signatures — this breaks compatibility with whatever partner/client is calling it.

Real-world enterprise scenario

A travel agency's booking portal calls your `WebServiceRQ` method to search flights. The agency's system was built 10 years ago and still expects the exact same XML shape. If you change the shape of the request or response even slightly, their integration breaks — possibly without you knowing until a partner calls support.

Exercises

1. Find one existing SOAP request/response pair used in the project (log files, Postman collection, or a test) and read it end to end.
2. Convert that same request into an equivalent JSON shape, just on paper, to build the mental mapping between SOAP-XML and REST-JSON.

Mini quiz

1. What is SOAP, fundamentally?
2. Why do legacy industries (airlines, banks) still use SOAP?
3. True or False: SOAP requires ASP.NET's classic ASMX technology.

(Answers: 1. An XML message format/contract over HTTP. 2. Expensive-to-replace integrations with long-lived partners who still expect XML/SOAP. 3. False — SOAP is a protocol, not tied to

any specific implementation.)

Summary

SOAP is just XML-over-HTTP with a strict envelope format. Your company kept it because it's tied to how their systems (and possibly external flight partners) already communicate. Nothing magical — just a message format.

Preparing for Module 2

Next we'll look at **WCF** — the .NET framework that *defines* the contracts (`[ServiceContract]`, `[OperationContract]`) your company reuses, even though you're not running classic WCF hosting.

Module 2 — WCF Architecture (Only the Parts You Actually Use)

Learning Objectives

- Understand what WCF is and which piece of it your company actually uses.
- Understand why your company uses WCF *attributes* without WCF *hosting*.

Theory, explained simply

WCF (Windows Communication Foundation) is a full framework Microsoft built for building service-oriented applications. It has many pieces:

WCF Piece	What it does	Do you use it?
`[ServiceContract]` / `[OperationContract]`	Define what operations a service exposes	**Yes**
`[DataContract]` / `[DataMember]`	Define strongly-typed message shapes	Rarely (you mostly pass raw XML strings)
`ServiceHost`	Hosts/runs a WCF service	**No**
`.svc` files	IIS-specific WCF hosting files	**No**
Bindings (`NetTcpBinding`, `wsHttpBinding`, etc.)	Configure transport/protocol	**No** — SoapCore configures this for you
WCF Security / Transactions / Duplex	Advanced enterprise features	**No**

So: your company uses WCF's **contract/attribute model** (a clean, well-established way of *describing* a service interface) but runs it on **ASP.NET Core**, using **SoapCore** as the bridge that turns those attributes into a working SOAP endpoint. This is a very deliberate, modern choice — you get old WCF-style contracts on new, cross-platform ASP.NET Core infrastructure.

Why this concept exists

`[ServiceContract]`/`[OperationContract]` is simply a clean, declarative way to say "this interface is a service, and these methods are callable operations." It's the .NET equivalent of an interface annotation system, similar in spirit to how `[ApiController]` and `[HttpGet]` declare intent in Web API.

Where it appears in enterprise applications

Any legacy .NET enterprise system built between ~2006–2015 likely used full WCF. Many companies migrating those systems to ASP.NET Core keep the contracts (since rewriting them and every client is expensive) and swap only the hosting layer — which is exactly your company's approach via SoapCore.

How it relates to our company

```
[ServiceContract]
public interface IFlightService
{
    [OperationContract]
    XmlDocument WebServiceRQ(string strRQ);
}
```

This interface is the **contract**. `FlightService` is the **implementation**. SoapCore reads this contract via reflection and exposes it as a SOAP endpoint automatically — no `ServiceHost`, no `.svc` file, no XML config binding. Just:

```
builder.Services.AddScoped<IFlightService, FlightService>();
builder.Services.AddSoapCore();
// ...
app.UseSoapEndpoint<IFlightService>("/FlightService.asmx", new
SoapEncoderOptions());
```

Practical example

Think of it exactly like Web API controllers, side by side:

```
// Web API style you already know
[ApiController]
[Route("api/[controller]")]
public class FlightController : ControllerBase
{
    [HttpPost]
    public IActionResult Search(SearchRequest req) => Ok(_service.Search(req));
}

// WCF-contract + SoapCore style used here
[ServiceContract]
public interface IFlightService
{
    [OperationContract]
    XmlDocument WebServiceRQ(string strRQ);
}
```

Same purpose (expose an operation over HTTP), different declaration style and wire format.

Diagram

```
 [ServiceContract] interface  ⟶  SoapCore reflects over it  ⟶  SOAP endpoint
auto-generated
        (IFlightService)
(/FlightService.asmx)
```

Best practices

- When you need to understand a service's public "shape," go straight to the interface (`IFlightService`), not the implementation. It's the contract — the implementation is just details.
- Don't try to introduce `ServiceHost`/`.svc` patterns from WCF tutorials you find online — they don't apply here.

Common mistakes

- Googling "WCF tutorial" and following instructions about `web.config` bindings — irrelevant to your ASP.NET Core + SoapCore setup.
- Assuming `[ServiceContract]` requires classic WCF hosting to work — it doesn't, SoapCore replaces that entirely.

Real-world enterprise scenario

A new hire spends two days trying to configure `NetTcpBinding` in `web.config` because a WCF book told them to, before realizing the project has no `web.config` service model section at all — because SoapCore handles it in `Program.cs`. Knowing what your company does and doesn't use saves this wasted time.

Exercises

1. Open `Program.cs` (or `Startup.cs`) and find the `AddSoapCore()` / `UseSoapEndpoint<...>()` calls.
2. List every `[ServiceContract]` interface in the solution — this is your map of "services that exist."

Mini quiz

1. Which two WCF attributes does your company actually use?
2. What replaces `ServiceHost` in your architecture?
3. True or False: your project has `.svc` files.

(Answers: 1. `[ServiceContract]`, `[OperationContract]`. 2. SoapCore's `AddSoapCore()`/`UseSoapEndpoint()`. 3. False.)

Summary

WCF, in your world, is just a naming/attribute convention borrowed for its clean contract style. All the actual hosting, binding, and transport work is done by SoapCore on top of ASP.NET Core.

Preparing for Module 3

Next, we go one level deeper into `[ServiceContract]` itself — what it does under the hood and how SoapCore uses it.

Module 3 — `[ServiceContract]`

Learning Objectives

- Understand exactly what `[ServiceContract]` declares.
- Understand its (optional) properties and when they matter.

Theory, explained simply

`[ServiceContract]` is applied to an **interface**. It tells the runtime (in your case, SoapCore): "this interface describes a service boundary — treat its `[OperationContract]`-marked methods as externally callable operations."

```
[ServiceContract]
public interface IFlightService
{
    [OperationContract]
    XmlDocument WebServiceRQ(string strRQ);
}
```

Optional properties you may see:

- `Namespace` — the XML namespace used in the generated WSDL/SOAP messages (important for matching what a client expects).
- `Name` — overrides the contract's exposed name.

```
[ServiceContract(Namespace = "http://yourcompany.com/flights")]
public interface IFlightService { ... }
```

Why this concept exists

Without `[ServiceContract]`, there would be no declarative way to say "this interface, specifically, is the public API surface" versus any other internal interface in your codebase. It's a marker, just like `[ApiController]` marks a class as a Web API controller.

Where it appears in enterprise applications

Every WCF-descended service in an enterprise codebase starts with a `[ServiceContract]` interface. It's the very first thing you should locate when investigating an unfamiliar service.

How it relates to our company

When you're asked "how do I add a new SOAP service for HotelService?", step one is always: define a new `[ServiceContract]` interface, following the existing pattern.

```
[ServiceContract]
```

```
public interface IHotelService
{
    [OperationContract]
    XmlDocument WebServiceRQ(string strRQ);
}
```

Practical example

Compare to something you already know — an ASP.NET Core Web API interface used for DI:

```
public interface IHotelApiService  // just a normal interface
{
    HotelSearchResult Search(HotelSearchRequest req);
}
```

`[ServiceContract]` is the same idea, plus metadata that tells SoapCore "expose this."

Diagram

```
public interface IFlightService        <-- just a C# interface
      ▲
      | [ServiceContract]              <-- "expose this as a SOAP service"
      |
  SoapCore reads this via reflection at startup
```

Best practices

- Keep the `[ServiceContract]` interface minimal and stable — it's a public contract; treat changes to it like breaking API changes.
- Match `Namespace` exactly to what any WSDL/existing client already expects (Module 7 covers WSDL).

Common mistakes

- Forgetting `[ServiceContract]` on the interface and only annotating methods — SoapCore won't recognize the service at all.
- Renaming the interface without checking whether any WSDL or client depends on the old name/namespace.

Real-world enterprise scenario

A teammate adds a new method to `IFlightService` but forgets it needs `[OperationContract]` (next module) — the method silently isn't exposed over SOAP, and they spend an hour debugging "why is my new method not callable."

Exercises

1. Open every `[ServiceContract]` interface in the solution and note their namespaces.
2. Try (in a scratch project, not production code) removing `[ServiceContract]` and see that SoapCore stops recognizing the service.

Mini quiz

1. What does `[ServiceContract]` mark?
2. What's one optional property of `[ServiceContract]` and what does it control?
3. True or False: You can put `[ServiceContract]` on a class instead of an interface.

(Answers: 1. An interface as a service's public contract. 2. `Namespace` — controls the XML namespace in the SOAP messages/WSDL. 3. It's conventionally applied to interfaces; that's the pattern this codebase uses — stick to it.)

Summary

`[ServiceContract]` is the "this is a service" flag on an interface. It's simple, but everything else (routing, WSDL generation, SOAP exposure) depends on it being there.

Preparing for Module 4

Now that the whole interface is marked as a service, we need to mark *which methods* are actually callable — that's `[OperationContract]`.

Module 4 — `[OperationContract]`

Learning Objectives

- Understand what `[OperationContract]` does at the method level.
- Understand why your company's operations are almost always shaped as `string in -> XmlDocument out`.

Theory, explained simply

`[OperationContract]` marks an individual method inside a `[ServiceContract]` interface as an operation that's callable from outside (i.e., exposed over SOAP).

```
[ServiceContract]
public interface IFlightService
{
    [OperationContract]
    XmlDocument WebServiceRQ(string strRQ);
}
```

Only methods marked `[OperationContract]` are exposed. Any other method on the interface (or any method in the implementing class not on the interface) is invisible to SOAP clients.

Why this concept exists

Interfaces can have many methods; not all of them are meant to be public network operations. `[OperationContract]` gives fine-grained control — you might have helper methods on the interface for internal reasons (rare, but possible) that you don't want exposed.

Where it appears in enterprise applications

This is the method-level equivalent of `[HttpGet]`/`[HttpPost]` in Web API controllers — a way of saying "this specific method is a public entry point."

How it relates to our company

Almost every operation in your services follows this exact shape:

```
[OperationContract]
XmlDocument WebServiceRQ(string strRQ);
```

This is a deliberate, generic design: **one operation per service, taking a raw XML request string, returning a raw XML document**. This is sometimes called a "single envelope" or "generic operation" pattern — it lets the actual business routing happen *inside* the XML (e.g., a `<RequestType>SearchFlights</RequestType>` node) rather than through separate strongly-typed methods. You'll see this a lot in older enterprise integration systems, especially ones that evolved over many years without wanting to keep changing the public contract.

Practical example

```
public class FlightService : IFlightService
{
    public XmlDocument WebServiceRQ(string strRQ)
    {
        XmlDocument xdIn = new XmlDocument();
        xdIn.LoadXml(strRQ);

        string requestType =
xdIn.DocumentElement.SelectSingleNode("//RequestType")?.InnerText;

        return requestType switch
        {
            "SearchFlights" => HandleSearch(xdIn),
            "BookFlight"    => HandleBooking(xdIn),
            _               => BuildErrorResponse("Unknown request type")
        };
    }
}
```

Notice: the *WCF contract* only exposes one operation, but internally it behaves like a small router to many "virtual operations." This is exactly why reading the XML carefully (Module 10) matters so much in this codebase.

Diagram

```
Client SOAP call
      │
      ▼
[OperationContract] WebServiceRQ(string strRQ)
      │
      ▼
   Parse strRQ → find <RequestType>
      │
      ├── "SearchFlights" → HandleSearch()
      ├── "BookFlight"    → HandleBooking()
      └── default         → error XML
```

Best practices

- When investigating a bug, first find which `<RequestType>` (or similar discriminator) the failing request used, then jump straight to that branch.
- Keep the "router" (top-level `WebServiceRQ`) as thin as possible — one line to look up the type, one line to dispatch. Business logic belongs in the handler methods.

Common mistakes

- Adding a new "operation" as a brand-new interface method instead of a new internal branch — this breaks the established pattern and can break backward-compatibility expectations for existing SOAP clients that only know one method.
- Forgetting `[OperationContract]` on a new interface method and wondering why it's not reachable.

Real-world enterprise scenario

A partner system starts sending a new `<RequestType>CancelFlight</RequestType>` that nobody implemented a handler for yet. Your `default` branch returns "Unknown request type," and support gets a ticket. Understanding this router pattern lets you diagnose that in minutes instead of hours.

Exercises

1. Find the `switch`/`if-else` block (or similar) inside a real service's `WebServiceRQ` implementation and list every request type it currently handles.
2. Pick one request type and trace its handler function from start to SQL access.

Mini quiz

1. What happens to an interface method without `[OperationContract]`?
2. Why does the company expose one generic operation instead of many strongly-typed ones?
3. Where does the "real" routing between different business actions happen?

(Answers: 1. It's not exposed over SOAP at all. 2. To keep the public WCF contract stable while still supporting many internal request types — common in long-lived enterprise integrations. 3. Inside the method body, based on parsing the XML request, not in the WCF contract itself.)

Summary

`[OperationContract]` marks which methods are network-callable. In your codebase there's usually just one per service, and it internally behaves like a mini-router based on the XML payload's request type.

Preparing for Module 5

Next: Data Contracts — and why you'll see very little of them here, since your company favors raw XML over strongly-typed messages.

Module 5 — Data Contracts (Only What's Necessary)

Learning Objectives

- Understand what `[DataContract]`/`[DataMember]` are for.
- Understand why your company mostly *doesn't* use them, and what it uses instead.

Theory, explained simply

In "classic" WCF, `[DataContract]` and `[DataMember]` let you define strongly-typed message classes, similar to how you'd define a POCO/DTO in Web API:

```
[DataContract]
public class FlightSearchRequest
{
    [DataMember]
    public string Origin { get; set; }

    [DataMember]
    public string Destination { get; set; }
}
```

WCF would then auto-serialize/deserialize this to/from XML for you.

Why this concept exists

Data Contracts exist so services can exchange strongly-typed, self-describing messages instead of hand-parsed strings — safer, more maintainable, and better tooling support (auto-generated client code, validation, etc.).

Where it appears in enterprise applications

Most textbook WCF services use Data Contracts heavily. If you take a generic WCF course, this will be a huge chunk of it — probably too much, for your case.

How it relates to our company

Your company deliberately does NOT use this pattern for these services. Instead of:

```
[OperationContract]
FlightSearchResponse Search(FlightSearchRequest request);
```

you have:

```
[OperationContract]
XmlDocument WebServiceRQ(string strRQ);
```

This means: no `[DataContract]`, no `[DataMember]`, no automatic (de)serialization. Everything is manually parsed with `XmlDocument` (Module 10). This is common in older, integration-heavy systems where the exact XML shape is dictated by an external partner/spec (e.g., an airline GDS format) that doesn't map cleanly onto auto-generated C# classes — so it's easier to just work with the raw XML directly.

Practical example

If you ever see a `[DataContract]` in this codebase, it's likely a small, internal-only helper class — not part of the actual wire contract for `WebServiceRQ`. Don't assume it behaves like the request/response shape; check how it's actually used.

Diagram

```
Classic WCF style (NOT your company):
  [DataContract] class  <——auto-serialized——>  XML

Your company's style:
  string strRQ  <——manually parsed with XmlDocument——>  business logic
```

Best practices

- Don't introduce `[DataContract]` classes to "modernize" the request/response shape unless the whole team agrees — it would break the established, working pattern and possibly external client expectations.
- If you do see typed classes in the codebase, check whether they're just internal helpers, not part of the actual SOAP contract.

Common mistakes

- Assuming there's a "hidden" typed contract you're missing — there usually isn't; it really is just an XML string in, XML document out.
- Wasting time learning `[DataContract]`/`[DataMember]` deeply for this project — it's low priority here.

Real-world enterprise scenario

A new hire, coming from a Web API background, spends a day trying to find "the DTO" for a flight search request before realizing the "DTO" is just an XML string built manually with `XmlDocument`/`StringBuilder`. This module exists so that doesn't happen to you.

Exercises

1. Search the solution for `[DataContract]` and confirm whether any exist, and if so, what they're used for.
2. Confirm for yourself that `IFlightService` and similar interfaces contain no typed request/response classes.

Mini quiz

1. What do `[DataContract]`/`[DataMember]` normally do in WCF?
2. Does your company use them for the main SOAP contracts?
3. What does your company use instead?

(Answers: 1. Define strongly-typed, auto-(de)serialized message shapes. 2. No, generally not for

the main services. 3. Raw XML strings parsed manually with `XmlDocument`.)

Summary

Data Contracts are a WCF feature you can mostly skip. Your company's services pass raw XML strings and documents instead of typed classes — keep that firmly in mind so you don't go looking for something that isn't there.

Preparing for Module 6

Now that you know *why* everything is raw XML, let's actually learn to read and build SOAP messages properly.

Module 6 — SOAP Messages

Learning Objectives

- Be able to read a full SOAP envelope confidently.
- Understand how your `strRQ` (inner request XML) relates to the outer SOAP envelope.

Theory, explained simply

There are actually **two layers of XML** in your system:

1. **The outer SOAP envelope** — generated/consumed automatically by SoapCore. You basically never touch this by hand.
2. **The inner request/response XML** — the actual string (`strRQ`) that your business code parses and builds manually.

Full example of what goes over the wire:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <WebServiceRQ xmlns="http://tempuri.org/">
      <strRQ>
        &lt;Request&gt;
          &lt;RequestType&gt;SearchFlights&lt;/RequestType&gt;
          &lt;User&gt;jdoe&lt;/User&gt;
          &lt;Origin&gt;TUN&lt;/Origin&gt;
          &lt;Destination&gt;CDG&lt;/Destination&gt;
        &lt;/Request&gt;
      &lt;/strRQ&gt;
    </WebServiceRQ>
  </soap:Body>
</soap:Envelope>
```

Notice the inner XML is **HTML-escaped** (`<`, `>`) because it's really just a string parameter — this is exactly your `strRQ`. Once SoapCore unwraps the SOAP envelope and hands you the plain string, `strRQ` looks like:

```
<Request>
  <RequestType>SearchFlights</RequestType>
```

```
  <User>jdoe</User>
  <Origin>TUN</Origin>
  <Destination>CDG</Destination>
</Request>
```

This is the XML you actually work with day to day.

Why this concept exists

SOAP defines the outer "transport envelope" so any SOAP-aware client/tool can talk to any SOAP-aware server without knowing implementation details. The inner XML is entirely up to your company/domain — it's a private, custom format layered inside a generic transport.

Where it appears in enterprise applications

This "generic envelope + custom inner payload" pattern is extremely common in legacy B2B integrations, particularly with GDS/airline systems, where the inner schema was defined decades ago and is shared across many client companies.

How it relates to our company

When debugging, always ask: "Am I looking at the outer SOAP envelope (rarely relevant, mostly SoapCore's job) or the inner request XML (this is where 95% of your work happens)?"

Practical example

```
public XmlDocument WebServiceRQ(string strRQ)
{
    // strRQ here is ALREADY unwrapped from the SOAP envelope by SoapCore.
    // It is the inner <Request>...</Request> XML as a plain string.
    XmlDocument xdIn = new XmlDocument();
    xdIn.LoadXml(strRQ);
    ...
}
```

Diagram

```
┌──────────────────────────────────────────┐
│ SOAP Envelope (handled by SoapCore)      │
│  ┌──────────────────────────────────────┐│
│  │ strRQ (your job, plain C# string)    ││
│  │  <Request>                           ││
│  │     <RequestType>...</RequestType>   ││
│  │     ...                              ││
│  │  </Request>                          ││
│  └──────────────────────────────────────┘│
└──────────────────────────────────────────┘
```

Best practices

- When capturing traffic for debugging, always identify and extract just the inner XML — that's the part worth staring at.
- Keep a few sample inner-XML requests/responses saved locally for each request type; they're worth more than documentation.

Common mistakes

- Trying to manually parse the outer SOAP envelope yourself — SoapCore already does this; you should never need `<soap:Envelope>` parsing code in `FlightService`.
- Confusing escaped XML (`&lt;Request&gt;`) in logs with malformed XML — it's expected; that's just XML-inside-a-string-inside-XML.

Real-world enterprise scenario

A support ticket includes a raw HTTP capture of a failing SOAP call. A junior developer panics because the XML "looks broken" (full of `&lt;`). Understanding that this is just an escaped inner string saves confusion — you decode/unescape it, and it reads like normal XML.

Exercises

1. Take a full outer SOAP envelope example and manually pull out the "true" inner request XML by mentally un-escaping it.
2. Do the reverse: take an inner XML request and write out what the full outer SOAP envelope would look like.

Mini quiz

1. Which XML layer does SoapCore handle for you?
2. Which XML layer do you write code to parse yourself?
3. Why is the inner XML escaped in the raw wire format?

(Answers: 1. The outer SOAP envelope. 2. The inner request/response XML (`strRQ`). 3. Because it's transmitted as a string value inside another XML document, so </> must be escaped to avoid breaking the outer XML structure.)

Summary

There are two XML layers — outer (SoapCore's job) and inner (your job). Almost all your day-to-day work is reading/writing the inner XML.

Preparing for Module 7

Next: WSDL — how a SOAP service describes itself to the outside world, and why it matters even though you rarely edit it by hand.

Module 7 — WSDL

Learning Objectives

- Understand what a WSDL is and what it's used for.
- Know how to view your service's WSDL and what to look for.

Theory, explained simply

WSDL (Web Services Description Language) is an XML document that describes a SOAP service: its operations, the shape of its messages, and where to find it (endpoint URL). Think of it as the SOAP equivalent of an OpenAPI/Swagger document for REST APIs, which you may have already seen with ASP.NET Core Web API.

You typically get a WSDL by appending `?wsdl` to a SOAP endpoint URL:

```
https://yourapp.com/FlightService.asmx?wsdl
```

Why this concept exists

WSDL lets any SOAP client — regardless of language/platform — generate a proxy/client automatically, without a human reading documentation. This was a big selling point of SOAP in the 2000s: strong, machine-readable contracts.

Where it appears in enterprise applications

Any client integrating with a SOAP service (an airline partner, a bank, another department) will usually point their tooling at your WSDL URL once, generate a client proxy, and call your service like a local method. If your contract changes in a breaking way, their generated proxy breaks.

How it relates to our company

SoapCore **auto-generates the WSDL** from your `[ServiceContract]`/`[OperationContract]` definitions — you don't hand-write it. This is exactly why the `[ServiceContract(Namespace = ...)]` value (Module 3) matters: it directly controls what shows up in the generated WSDL, which external clients may depend on.

Since your operations are just `WebServiceRQ(string strRQ) : XmlDocument`, the WSDL for your services is intentionally generic — it says "this operation takes a string and returns an XML document," not the full business shape (that's hidden inside the string). This is a trade-off: less type-safety in the WSDL, but a stable public contract even as internal request types evolve.

Practical example

```
GET https://yourapp.com/FlightService.asmx?wsdl
```

Opening this in a browser or Postman shows XML describing:

- The service name and namespace
- The `WebServiceRQ` operation
- The `strRQ` parameter and `XmlDocument` return type

Diagram

```
[ServiceContract] + [OperationContract]
              │
              ▼
     SoapCore generates WSDL automatically
              │
              ▼
```

```
External client reads WSDL → generates proxy → calls your service
```

Best practices

- Before changing a `[ServiceContract]`'s namespace or an operation's name, check if any known external client relies on the current WSDL.
- Keep a saved copy of the current WSDL for each service so you can diff it before/after a change.

Common mistakes

- Editing WSDL output expectations without realizing it's auto-generated from your C# contract — there's no separate WSDL file to hand-edit.
- Renaming a service interface casually, not realizing it changes the generated WSDL's service name, potentially breaking existing client proxies.

Real-world enterprise scenario

An external travel partner's system suddenly can't connect after a deployment. Investigation shows the `[ServiceContract]` namespace was changed as part of a "cleanup," which changed the WSDL, which broke their previously-generated client proxy. This is why WSDL, even though auto-generated, is treated as a fragile public contract.

Exercises

1. Fetch the WSDL for one of your services locally (dev environment) and read through its structure.
2. Identify the operation name and namespace inside the WSDL and match them back to the `[ServiceContract]`/`[OperationContract]` attributes in code.

Mini quiz

1. What does WSDL describe?
2. Who/what generates your WSDL?
3. What C# attribute property most directly affects the WSDL's namespace?

(Answers: 1. A SOAP service's operations, message shapes, and endpoint. 2. SoapCore, automatically, from your contract attributes. 3. `[ServiceContract(Namespace = ...)]`.)

Summary

WSDL is the auto-generated "API description" for your SOAP service. You don't write it by hand, but changes to your contract attributes can change it — and that can break external clients.

Preparing for Module 8

Now let's zoom out and look specifically at how SoapCore ties everything from Modules 2–7

together at runtime.

Module 8 — How SoapCore Uses WCF Contracts

Learning Objectives

- Understand the full startup pipeline: from `Program.cs` to a working SOAP endpoint.
- Be able to add a brand-new SOAP service to the project confidently.

Theory, explained simply

SoapCore is a community library that bridges classic WCF-style contracts (`[ServiceContract]`/`[OperationContract]`) with ASP.NET Core's hosting model (the same `Program.cs`/middleware pipeline you already know from Web API). At startup, it:

1. Registers your service implementation in DI (`AddScoped<IFlightService, FlightService>()`).
2. Registers the SoapCore services (`AddSoapCore()`).
3. Maps a middleware endpoint (`UseSoapEndpoint<IFlightService>(...)`) that listens on a specific URL, parses incoming SOAP envelopes, invokes the matching `[OperationContract]` method via reflection, and wraps the return value back into a SOAP envelope.

Why this concept exists

Companies wanted to keep decades-old WCF-shaped contracts (and the client integrations built against them) while modernizing the actual hosting to ASP.NET Core — for cross-platform support, better performance, and to get away from IIS/.NET Framework-only WCF hosting. SoapCore is the glue that makes that possible.

Where it appears in enterprise applications

Any .NET shop migrating legacy WCF services to ASP.NET Core without wanting to force every client to migrate to REST/JSON at the same time uses a tool like SoapCore (or writes similar custom middleware).

How it relates to our company

This is *exactly* your architecture. Here's the full picture, end-to-end, using patterns you already know from ASP.NET Core middleware/DI:

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<IFlightService, FlightService>();
builder.Services.AddSoapCore();

var app = builder.Build();
```

```
app.UseRouting();
app.UseSoapEndpoint<IFlightService>(
    "/FlightService.asmx",
    new SoapEncoderOptions(),
    SoapSerializer.XmlSerializer
);

app.Run();
```

Because you already understand ASP.NET Core middleware, think of `UseSoapEndpoint<IFlightService>(...)` as conceptually similar to `app.MapControllers()` — it's wiring up a piece of middleware that knows how to route incoming requests to the right place, just for SOAP instead of REST routing conventions.

Practical example — adding a brand-new service, step by step

1. Define the contract:

```
[ServiceContract]
public interface IHotelService
{
    [OperationContract]
    XmlDocument WebServiceRQ(string strRQ);
}
```

1. Implement it:

```
public class HotelService : IHotelService
{
    public XmlDocument WebServiceRQ(string strRQ) { /* parse, route, respond
*/ }
}
```

1. Register it:

```
builder.Services.AddScoped<IHotelService, HotelService>();
```

1. Expose it:

```
app.UseSoapEndpoint<IHotelService>("/HotelService.asmx", new
SoapEncoderOptions());
```

Diagram

```
Program.cs
   │
   ├─ AddScoped<IFlightService, FlightService>()   (DI registration — same as
Web API)
   ├─ AddSoapCore()                                (registers SoapCore
internals)
   │
   ▼
app.UseSoapEndpoint<IFlightService>("/FlightService.asmx", ...)
   │
   ▼
Incoming HTTP POST to /FlightService.asmx
   │
   ▼
SoapCore parses SOAP envelope → finds matching [OperationContract] → invokes
FlightService.WebServiceRQ(strRQ)
```

```
   │
   ▼
Return value wrapped back into a SOAP envelope → sent to client
```

Best practices

- When adding a new service, copy the pattern of an existing one exactly (contract → implementation → DI registration → `UseSoapEndpoint`) rather than improvising.
- Keep each service's URL path (`"/FlightService.asmx"`) consistent with naming conventions already used in the project, since external clients (and internal tooling) may rely on the exact path.

Common mistakes

- Forgetting to register the service in DI (`AddScoped<...>`) — SoapCore will fail to resolve the implementation at runtime.
- Forgetting `app.UseSoapEndpoint<T>()` entirely — the contract and implementation exist, but there's no route to reach them.
- Registering the same endpoint path twice for two different services.

Real-world enterprise scenario

A ticket asks you to add a `CarRentalService`. Following this exact four-step pattern (contract, implementation, DI, endpoint mapping) lets you deliver it confidently in an afternoon, matching the existing architecture instead of inventing a new pattern.

Exercises

1. Find every `UseSoapEndpoint<...>()` call in `Program.cs`/`Startup.cs` and match each to its `[ServiceContract]` interface.
2. On a scratch branch, add a new dummy `IPingService` following the four-step pattern, and confirm it responds via a SOAP client (e.g., Postman or SoapUI).

Mini quiz

1. What are the four steps to add a new SOAP service in this codebase?
2. What ASP.NET Core concept is `UseSoapEndpoint<T>()` most similar to?
3. What happens if you forget to register the service implementation in DI?

(Answers: 1. Define contract, implement it, register in DI, map with `UseSoapEndpoint`. 2. Middleware/endpoint mapping, similar to `MapControllers()`. 3. SoapCore throws a resolution error at runtime when a request comes in.)

Summary

SoapCore reads your WCF-style contracts and, using standard ASP.NET Core DI and middleware concepts you already know, exposes them as working SOAP endpoints — no `.svc`, no

`ServiceHost`, no XML config.

Preparing for Module 9

Now that you understand the full pipeline, let's build your actual reading skill: how to approach an unfamiliar legacy SOAP service file for the first time.

Module 9 — Reading Legacy Enterprise SOAP Code

Learning Objectives

- Develop a repeatable checklist for understanding any unfamiliar service in this codebase.
- Build confidence reading dense, "old-style" enterprise C#/XML code.

Theory, explained simply

Legacy enterprise SOAP services tend to look intimidating because of density — long methods, lots of manual XML manipulation, minimal comments, string-based branching. But the underlying structure is almost always the same, once you know what to look for.

Why this concept exists

Enterprise codebases accumulate years of incremental changes by many different developers. Nobody sits down and "designs" it fresh — it evolves. Learning to read *existing* patterns quickly is a more valuable and realistic skill than trying to understand "ideal" architecture from a textbook.

Where it appears in enterprise applications

This skill transfers far beyond SOAP — reading unfamiliar, dense legacy code is a core skill in any long-lived enterprise system (COBOL, old Java EE, classic ASP, VB6-turned-C#, etc.).

How it relates to our company — your reading checklist

When you open an unfamiliar service file, follow this order:

1. **Find the contract.** Locate the `[ServiceContract]` interface — this tells you the service's name and its single (or few) operation(s).
2. **Find the implementation's entry point.** Usually `WebServiceRQ(string strRQ)`.
3. **Find the dispatcher.** Look for the `if`/`switch` on something like `<RequestType>` — this is your "table of contents" for everything this service can do.
4. **Pick one branch.** Don't try to read the whole file at once — trace exactly one request type from start to finish.
5. **Find the SQL.** Most branches eventually call into ADO.NET (which you already know) to hit SQL Server.
6. **Find the response builder.** See how the XML response is constructed (Module 10) and returned.

Practical example

```
public XmlDocument WebServiceRQ(string strRQ)
{
    XmlDocument xdIn = new XmlDocument();
    xdIn.LoadXml(strRQ);

    string reqType =
xdIn.DocumentElement.SelectSingleNode("//RequestType")?.InnerText;

    switch (reqType)
    {
        case "SearchFlights": return HandleSearchFlights(xdIn);   // <- step 4:
pick this one
        case "BookFlight":    return HandleBookFlight(xdIn);
        default:              return BuildErrorXml("Unknown request type: " +
reqType);
    }
}

private XmlDocument HandleSearchFlights(XmlDocument xdIn)
{
    string origin =
xdIn.DocumentElement.SelectSingleNode("//Origin")?.InnerText;
    string destination =
xdIn.DocumentElement.SelectSingleNode("//Destination")?.InnerText;

    // step 5: SQL access, using ADO.NET you already know
    var flights = _flightRepository.SearchFlights(origin, destination);

    // step 6: build XML response
    return BuildSearchResponseXml(flights);
}
```

Diagram

```
1. [ServiceContract] interface  → what's the service's name/purpose?
2. WebServiceRQ entry point     → where does every request start?
3. Dispatcher (switch/if)       → what are all the possible actions?
4. One branch                   → trace exactly this, ignore the rest for now
5. SQL access                   → what data does this touch?
6. Response builder             → what XML comes back, and in what shape?
```

Best practices

- Never try to understand an entire service file in one pass. Trace one request type end-to-end first; the rest follow the same pattern.
- Keep a running personal notes file per service: "what request types exist, what each does, which tables it touches." This becomes gold during your internship.

Common mistakes

- Getting overwhelmed trying to read top-to-bottom like a story — enterprise SOAP code is not written to be read that way; it's written to be *dispatched into*.
- Ignoring the `[ServiceContract]` interface and jumping straight into a 500-line implementation file — always start with the contract.

Real-world enterprise scenario

You're asked to fix a bug in "flight search returning wrong prices." Using the checklist: you go to `IFlightService` → `FlightService.WebServiceRQ` → find `case "SearchFlights"` → trace `HandleSearchFlights` → find the SQL query or business calculation for price → fix it. Total time: 20 minutes, instead of hours reading the whole file.

Exercises

1. Pick the largest/scariest-looking service file in the codebase. Apply the 6-step checklist to just one request type in it.
2. Write a one-paragraph summary, in your own words, of what that one request type does, from XML request to XML response.

Mini quiz

1. What should you always find first when reading an unfamiliar service?
2. What's the "table of contents" for a service's capabilities usually called in this codebase?
3. Why shouldn't you read a legacy SOAP service file top-to-bottom like a story?

(Answers: 1. The `[ServiceContract]` interface. 2. The request-type dispatcher (switch/if on `<RequestType>` or similar). 3. Because it's structured as a dispatcher with independent branches, not a single linear narrative — tracing one branch at a time is far more effective.)

Summary

Legacy SOAP code is less scary once you have a repeatable checklist: contract → entry point → dispatcher → one branch → SQL → response. This is now your go-to method for any unfamiliar service.

Preparing for Module 10

Time to get hands-on with the tool you'll use constantly: `XmlDocument`, and the exact patterns your company uses to read and build XML.

Module 10 — XML in Enterprise Applications

Learning Objectives

- Be fluent in `XmlDocument`, `SelectSingleNode`, `SelectNodes`, `CreateElement`, `AppendChild`.
- Be able to both **read** an incoming request and **build** an outgoing response confidently.

Theory, explained simply

`XmlDocument` is a classic .NET Framework/.NET class (from `System.Xml`) representing an in-memory XML tree you can navigate and modify. Since your company doesn't use typed Data

Contracts (Module 5), this is the *primary tool* you'll use daily.

Key methods: | Method | Purpose | |---|---| | `LoadXml(string)` | Parse a string into an `XmlDocument` | | `SelectSingleNode(xpath)` | Find one node using an XPath expression | | `SelectNodes(xpath)` | Find multiple matching nodes | | `CreateElement(name)` | Create a new XML element | | `AppendChild(node)` | Attach a node as a child of another | | `InnerText` | Get/set the text content of a node | | `InnerXml` | Get/set the raw XML content of a node |

Why this concept exists

Before typed serialization became the default, and in systems where the exact wire format must match an external spec exactly, working directly with the XML DOM gives full, precise control over the output — every element, attribute, and namespace exactly as required, with no surprises from auto-serialization.

Where it appears in enterprise applications

Any integration where the XML schema is dictated by a third party (banks, airlines, government systems) that predates or doesn't map cleanly to modern serializers tends to use this manual approach.

How it relates to our company

This is used **everywhere** in your services. Reading:

```
XmlDocument xdIn = new XmlDocument();
xdIn.LoadXml(strRQ); // NOTE: your example used InnerXml — LoadXml is the more
common/safe API; both work similarly here

string user = xdIn.DocumentElement
                  .SelectSingleNode("//User")
                  .InnerText;
```

Building a response follows the mirror-image pattern:

```
XmlDocument xdOut = new XmlDocument();
XmlElement root = xdOut.CreateElement("Response");
xdOut.AppendChild(root);

XmlElement statusNode = xdOut.CreateElement("Status");
statusNode.InnerText = "OK";
root.AppendChild(statusNode);

XmlElement flightsNode = xdOut.CreateElement("Flights");
root.AppendChild(flightsNode);

foreach (var flight in flights)
{
    XmlElement flightNode = xdOut.CreateElement("Flight");

    XmlElement numberNode = xdOut.CreateElement("Number");
    numberNode.InnerText = flight.Number;
    flightNode.AppendChild(numberNode);

    XmlElement priceNode = xdOut.CreateElement("Price");
    priceNode.InnerText = flight.Price.ToString();
    flightNode.AppendChild(priceNode);
```

```
    flightsNode.AppendChild(flightNode);
}

return xdOut;
```

Practical example — a note on `//` in XPath

`SelectSingleNode("//User")` uses `//`, which means "search anywhere in the document," not just direct children. This is convenient but can be a trap: if the XML ever contains more than one `<User>` element at different nesting levels (e.g., inside both a `<Passenger>` and a `<BookingAgent>` block), `//User` will match the *first* one found in document order — which might not be the one you want. Prefer a fully-qualified path (e.g., `/Request/User`) when precision matters.

Diagram

```
strRQ (string)
   │  LoadXml()
   ▼
XmlDocument (in-memory tree)
   │  SelectSingleNode("/Request/User")
   ▼
"jdoe" (InnerText)


Building the response:
CreateElement("Response") → AppendChild → CreateElement("Status") → InnerText =
"OK" → AppendChild
   → results in a full XmlDocument, returned as-is from WebServiceRQ
```

Best practices

- Always null-check `SelectSingleNode` results (`?.InnerText`) — a missing node returns `null`, and calling `.InnerText` on `null` throws a `NullReferenceException`.
- Prefer specific XPath (`/Request/Origin`) over `//Origin` when the document could ever contain nested/duplicate element names.
- Build response XML in a small, dedicated helper method per response "shape" — keeps `WebServiceRQ` readable.

Common mistakes

- `xdIn.DocumentElement.SelectSingleNode("//User").InnerText` without a null check — a very common source of "random" `NullReferenceException`s in production when a partner sends a slightly malformed request.
- Mixing up `InnerText` (text content only) and `InnerXml` (raw XML, including child tags) — using the wrong one silently produces double-encoded or missing data.
- Forgetting to append a newly created element to its parent (`AppendChild`) — the element exists in memory but never appears in the final XML.

Real-world enterprise scenario

A partner occasionally omits the `<User>` element entirely in malformed requests. Because the code does `SelectSingleNode("//User").InnerText` without a null check, the service throws an unhandled exception and returns a generic 500 error instead of a helpful "missing User field" SOAP fault. Fixing this (add the null check, return a proper error XML) is a very realistic, very common first ticket for a junior developer.

Exercises

1. Take a sample request XML and write the `XmlDocument` code to extract three different fields from it, using safe null-conditional access.
2. Write a small standalone C# console snippet that builds a `<Response><Status>OK</Status></Response>` XML document from scratch and prints it.

Mini quiz

1. What's the difference between `InnerText` and `InnerXml`?
2. Why is `//FieldName` sometimes risky to use in `SelectSingleNode`?
3. What happens if you call `.InnerText` on the result of a `SelectSingleNode` that found nothing?

(Answers: 1. `InnerText` = text content only; `InnerXml` = raw XML markup including child elements. 2. It searches the entire document, so it can accidentally match a same-named element in an unrelated part of the tree. 3. `NullReferenceException`, because `SelectSingleNode` returns `null` when nothing matches.)

Summary

`XmlDocument` is your bread-and-butter tool in this codebase. Reading is mostly `SelectSingleNode(...).InnerText` (with null checks!); writing is mostly `CreateElement` + `AppendChild` + `InnerText`.

Preparing for Module 11

Now let's connect everything into one full, realistic trace: a request coming in from the Web Forms frontend, through the service, to SQL Server, and back.

Module 11 — Following a Request from Controller to Service (Full Trace)

Learning Objectives

- Trace one complete request end-to-end through every layer.
- Be able to do this yourself for any request type in the real codebase.

Theory, explained simply

This module doesn't introduce new concepts — it strings together everything from Modules 1–10 into one realistic, complete example, because in practice you'll always be reasoning about a *full trace*, not isolated pieces.

Why this concept exists

Understanding individual pieces (XML parsing, contracts, SoapCore) is necessary but not sufficient — real debugging and feature work requires holding the *entire path* in your head at once.

Where it appears in enterprise applications

Any layered enterprise system requires this "full trace" skill — it's identical in spirit to tracing a request through Controller → Service → Repository → Database in a Web API app, just with SOAP/XML instead of JSON/DTOs at the edges.

How it relates to our company — the full trace

Step 1 — Web Forms frontend builds the request

```
// Somewhere in a .aspx.cs code-behind
string requestXml =
    $"<Request>" +
        $"<RequestType>SearchFlights</RequestType>" +
        $"<Origin>{txtOrigin.Text}</Origin>" +
        $"<Destination>{txtDestination.Text}</Destination>" +
    $"</Request>";

var client = new FlightServiceClient(); // generated SOAP client / manual HTTP
call
XmlDocument response = client.WebServiceRQ(requestXml);
```

Step 2 — SOAP transport (SoapCore, Module 8) The client sends this wrapped in a full SOAP envelope (Module 6) via HTTP POST to `/FlightService.asmx`. SoapCore unwraps it and extracts `strRQ`.

Step 3 — Service entry point (Modules 3, 4)

```
public XmlDocument WebServiceRQ(string strRQ)
{
    XmlDocument xdIn = new XmlDocument();
    xdIn.LoadXml(strRQ);
    string reqType =
xdIn.DocumentElement.SelectSingleNode("/Request/RequestType")?.InnerText;
    return reqType == "SearchFlights" ? HandleSearchFlights(xdIn) :
BuildErrorXml("Unknown request");
}
```

Step 4 — Business logic + XML parsing (Module 10)

```
private XmlDocument HandleSearchFlights(XmlDocument xdIn)
{
    string origin =
xdIn.DocumentElement.SelectSingleNode("/Request/Origin")?.InnerText;
    string destination =
xdIn.DocumentElement.SelectSingleNode("/Request/Destination")?.InnerText;
```

```
    var flights = _flightRepository.SearchFlights(origin, destination); // Step
5
    return BuildSearchResponseXml(flights);                              // Step
6
}
```

Step 5 — Data access (ADO.NET, which you already know)

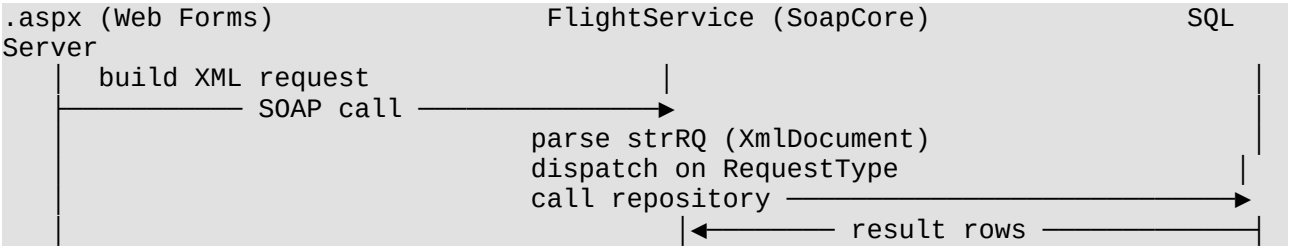
```
public List<Flight> SearchFlights(string origin, string destination)
{
    using var conn = new SqlConnection(_connectionString);
    using var cmd = new SqlCommand(
        "SELECT FlightNumber, Price FROM Flights WHERE Origin=@o AND
Destination=@d", conn);
    cmd.Parameters.AddWithValue("@o", origin);
    cmd.Parameters.AddWithValue("@d", destination);

    conn.Open();
    using var reader = cmd.ExecuteReader();
    var results = new List<Flight>();
    while (reader.Read())
        results.Add(new Flight { Number = reader.GetString(0), Price =
reader.GetDecimal(1) });
    return results;
}
```

Step 6 — Build XML response (Module 10)

```
private XmlDocument BuildSearchResponseXml(List<Flight> flights)
{
    var xdOut = new XmlDocument();
    var root = xdOut.CreateElement("Response");
    xdOut.AppendChild(root);
    foreach (var f in flights)
    {
        var flightNode = xdOut.CreateElement("Flight");
        var numberNode = xdOut.CreateElement("Number"); numberNode.InnerText =
f.Number;
        var priceNode = xdOut.CreateElement("Price"); priceNode.InnerText =
f.Price.ToString();
        flightNode.AppendChild(numberNode);
        flightNode.AppendChild(priceNode);
        root.AppendChild(flightNode);
    }
    return xdOut;
}
```

Step 7 — Back through SoapCore and to the browser SoapCore wraps the returned `XmlDocument` back into a SOAP envelope; the Web Forms code-behind reads the response `XmlDocument` and binds results (e.g., to a `GridView`).

Diagram



```
   │                              build XML response                              │
   │◄──────────── SOAP response ──────────┤                                       │
bind to GridView
```

Best practices

- When investigating any bug, draw (even on paper) this 7-step trace specific to the failing request type before touching code.
- Keep the "shape" of request/response XML for each request type documented somewhere the whole team can see — massively speeds up onboarding and debugging.

Common mistakes

- Jumping straight into SQL/business logic changes without first confirming, via the actual request XML, what's really being sent from the frontend.
- Forgetting the frontend is Web Forms, not a SPA — data typically comes from server controls (`txtOrigin.Text`), not client-side JS state.

Real-world enterprise scenario

A bug report says "search results are always empty." Tracing step by step, you find the Web Forms page is sending `<Origin>` as an airport *name* ("Paris") while SQL expects an airport *code* ("CDG") — a mismatch entirely invisible unless you actually walk the full request/response trace end-to-end.

Exercises

1. Pick a real request type in your codebase and write out all 7 steps yourself, with actual code from the repo at each step.
2. Identify, for that same request type, exactly which SQL table(s) are touched.

Mini quiz

1. List the 7 steps of a full request trace in this architecture.
2. Which step is where SoapCore's job ends and your business code begins?
3. Which existing skill (that you already have) applies directly in Step 5?

(Answers: 1. Frontend builds request → SOAP transport → service entry point → business logic/XML parsing → SQL data access → build XML response → SOAP transport back to frontend. 2. Right after unwrapping the SOAP envelope and calling `WebServiceRQ(strRQ)`. 3. ADO.NET.)

Summary

You now have a full, concrete, repeatable trace of a real request through this exact architecture — this is the single most valuable mental model for your internship.

Preparing for Module 12

Now that you can trace a request on paper, let's cover how to actually debug one when something breaks — locally and via captured traffic.

Module 12 — Debugging SOAP Applications

Learning Objectives

- Debug a SOAP request locally, step by step, in Visual Studio.
- Read raw HTTP/SOAP traffic to diagnose issues without a debugger attached.

Theory, explained simply

Debugging a SoapCore-based service is not fundamentally different from debugging any ASP.NET Core app — you can set breakpoints, run locally, and step through code exactly like you would with a Web API controller. The main extra skill is being comfortable reading the raw XML traffic, since the "input" isn't a friendly JSON body in a debugger watch window — it's an XML string you often need to inspect carefully.

Why this concept exists

In enterprise integrations, a large share of bugs come from **malformed, unexpected, or subtly different XML** sent by an external partner or an internal frontend — not from logic bugs in your C#. Being able to quickly capture and read that raw XML is often more valuable than stepping through code.

Where it appears in enterprise applications

Any team maintaining B2B XML integrations relies heavily on tools like Fiddler, Wireshark, browser dev tools (for calls originating from Web Forms via AJAX), or built-in request logging to capture the exact bytes going over the wire before making assumptions in code.

How it relates to our company

Practical debugging workflow for this codebase:

1. **Reproduce locally.** Run the ASP.NET Core project locally, point your Web Forms frontend (or a tool like Postman/SoapUI) at the local SOAP endpoint.
2. **Set a breakpoint at the top of `WebServiceRQ`.** This is your single most useful breakpoint — everything for that service flows through here.
3. **Inspect `strRQ` directly in the debugger's watch/immediate window.** Since it's just a string, you can copy it out and format it (many IDEs, or an online formatter, can pretty-print XML) to read it clearly.
4. **Step into the dispatcher**, confirm which branch is hit, and step through that branch specifically.

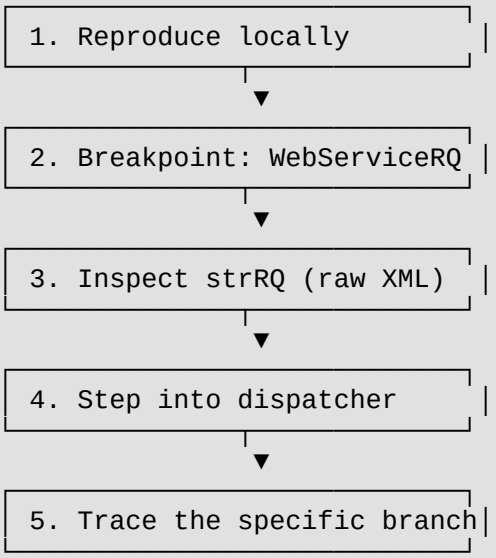
5. **If you can't reproduce locally**, get the raw captured SOAP request from logs, a proxy tool (Fiddler), or the partner/support ticket, and manually walk through the code with that exact XML in front of you (a "code trace on paper," as in Module 11) — you often don't even need to run the app to spot the bug.
6. **Check for SOAP Faults.** If SoapCore returns a SOAP fault (its equivalent of an unhandled exception), the fault's `<faultstring>` often has your original C# exception message — check application logs too.

Practical example

```
public XmlDocument WebServiceRQ(string strRQ)
{
    // 👇 put your breakpoint on this line
    XmlDocument xdIn = new XmlDocument();
    xdIn.LoadXml(strRQ); // hover/watch "strRQ" here to see the raw incoming XML
    ...
}
```

If `xdIn.LoadXml(strRQ)` itself throws, that's your first clue: the incoming XML is malformed (unescaped special characters, truncated content, wrong encoding) — the bug is upstream (the caller), not in your parsing logic.

Diagram



Best practices

- Always look at the raw XML before assuming the bug is in your C# logic — a huge share of "bugs" are actually bad/unexpected input.
- Add (or find and use) request/response logging around `WebServiceRQ` if it doesn't already exist — invaluable for diagnosing issues you can't reproduce locally.
- Pretty-print XML before reading it manually; dense single-line XML is very easy to misread.

Common mistakes

- Assuming a `NullReferenceException` inside `HandleSearchFlights` means your logic is wrong, when it's actually because an expected XML element was simply missing from this particular request (Module 10's null-check lesson).
- Debugging only the "happy path" locally with clean test data, missing edge cases (missing fields, unexpected encodings) that show up in production traffic.

Real-world enterprise scenario

Production support flags "SearchFlights sometimes fails, but works when we test it." You get the actual failing request from logs, notice `<Destination>` is completely empty (`<Destination></Destination>`) rather than missing, so your null-check on `SelectSingleNode` passes but the SQL query returns nothing, and downstream code doesn't handle "no results" gracefully. Comparing your local (clean) test XML against the actual failing XML side by side is what reveals this.

Exercises

1. Set a breakpoint on `WebServiceRQ` for any service, trigger a request from the frontend (or Postman/SoapUI), and inspect `strRQ` live.
2. Take a slightly broken/edge-case XML request (e.g., missing an element) and manually trace through the code on paper to predict what will happen before running it.

Mini quiz

1. What's usually the single most useful breakpoint location in these services?
2. What's often a faster first diagnostic step than attaching a debugger?
3. Where might useful error details show up if SoapCore returns a fault?

(Answers: 1. The top of `WebServiceRQ`. 2. Reading the raw captured request/response XML directly. 3. The fault's `<faultstring>` and/or the application's server-side logs.)

Summary

Debugging this architecture is mostly standard ASP.NET Core debugging, plus a strong habit of inspecting raw XML directly — most real bugs live in unexpected input shapes, not deep logic errors.

Preparing for Module 13

Finally, let's zoom all the way out and connect everything you've learned to the big picture of how enterprise architectures like this one are typically organized and why.

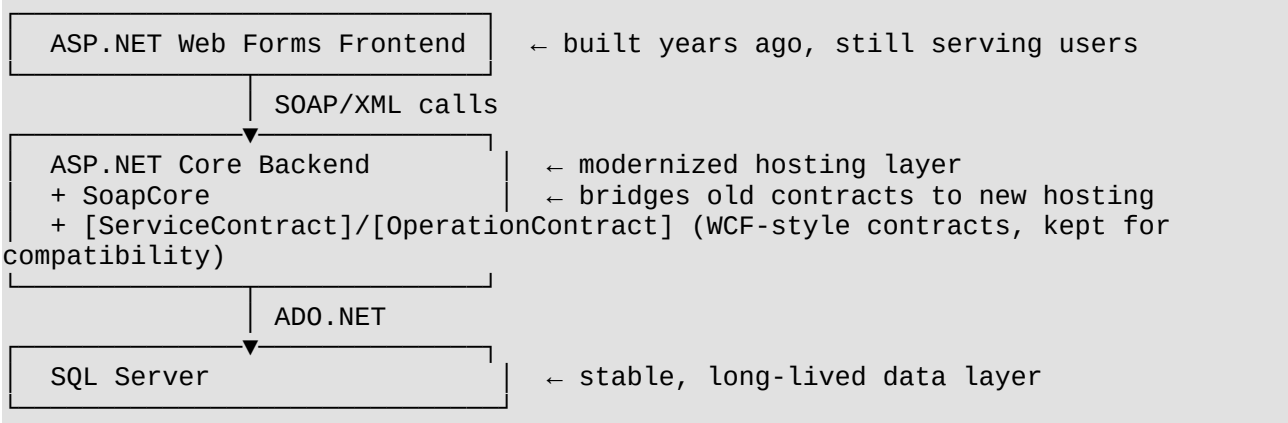
Module 13 — Understanding Enterprise Architectures Using SOAP

Learning Objectives

- Understand where this SOAP layer fits in a broader enterprise system.
- Be able to explain the architecture confidently to a teammate or in a standup.

Theory, explained simply

Enterprise systems are rarely "one clean technology." They're layered over years, and different layers often use whatever was the right (or available) choice *at the time they were built*, then get bridged together as needs evolve. Your company's stack is a great real-world example of exactly this:



Why this concept exists

Full rewrites are expensive and risky — every layer you rewrite is a chance to break something a client, partner, or internal team depends on. Enterprises instead **modernize incrementally**: swap the hosting/runtime (classic WCF → ASP.NET Core + SoapCore) while keeping the parts that are expensive to change (the contract shape, the XML schema, sometimes even the frontend).

Where it appears in enterprise applications

This "modernize the middle, keep the edges stable" pattern is extremely common: old frontend + modernized backend + old data contracts + new hosting, or vice versa. You'll see it in banks, airlines, insurance, telecom, and government software constantly.

How it relates to our company

You are joining at a specific, deliberate point in this modernization journey:

- **Frontend (Web Forms):** not yet modernized — you're learning it separately, and that's fine; it still works and still ships value.
- **Backend hosting:** modernized to ASP.NET Core.
- **Backend contracts:** kept as WCF-style (`[ServiceContract]`/`[OperationContract]`) via SoapCore, likely because

rewriting every consumer of these SOAP services (possibly external airline/GDS partners) would be far more expensive than modernizing the hosting alone.

- **Data layer:** SQL Server + ADO.NET, which you already know well.

Understanding *why* the architecture looks this way (not just *what* it looks like) will make you far more effective than someone who only memorizes the code patterns — you'll be able to reason about *why* a proposed change is risky or safe.

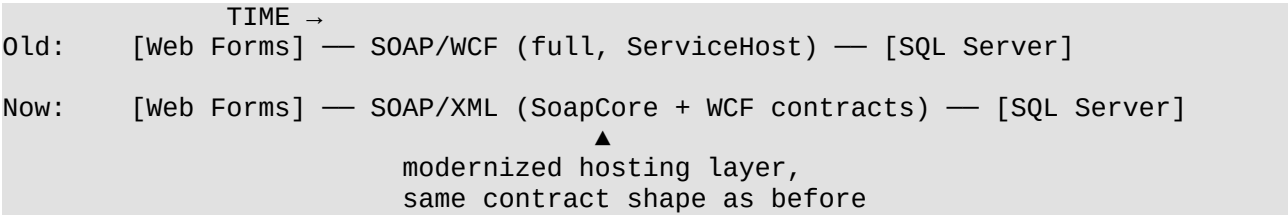
Practical example

If someone on your team proposes "let's just rewrite `IFlightService` as a REST API," the right questions to ask (now that you understand the architecture) are:

- Who calls this service today — internal Web Forms only, or external partners too?
- If external partners call it, can they even consume REST/JSON, or are they SOAP-only systems (e.g., older GDS integrations)?
- What's the migration cost/risk versus benefit?

This is exactly the kind of reasoning a senior engineer does before touching architecture — and now you can participate in that conversation.

Diagram



Best practices

- When proposing changes, always distinguish "changing the implementation" (safe, internal) from "changing the contract" (risky, potentially breaks external clients).
- Respect that this architecture is a deliberate trade-off, not an accident or "unfinished migration" — treat it with the same rigor you'd give a fully modern stack.

Common mistakes

- Assuming "old-looking" technology automatically means "bad" or "needs replacing" — often it means "stable, cheap to maintain, and serving its purpose well."
- Proposing a full rewrite without first mapping out who/what actually depends on the current contract.

Real-world enterprise scenario

A new engineer proposes replacing all `WebServiceRQ`-style SOAP operations with clean REST endpoints to "modernize" the codebase, not realizing three external airline partners have generated SOAP client proxies against the current WSDL that would all break simultaneously. Understanding

the architecture from this course lets you (politely) raise that risk in the discussion.

Exercises

1. Draw (on paper or in a tool) the full architecture diagram of your company's system, in your own words, labeling each layer and why it exists.
2. Write down, in a few sentences, what you'd tell a future new hire about *why* this architecture looks the way it does — practice explaining it out loud.

Mini quiz

1. Why do many enterprises modernize incrementally instead of doing full rewrites?
2. Which layer of your company's stack was modernized, and which was deliberately kept stable?
3. What question should you always ask before changing a service's contract?

(Answers: 1. Full rewrites are expensive and risky; incremental modernization limits blast radius. 2. Hosting layer modernized to ASP.NET Core + SoapCore; the WCF-style contract shape (and the Web Forms frontend) kept stable. 3. Who/what currently depends on this contract, and what would break if it changed?)

Summary

Your company's architecture isn't a half-finished migration — it's a deliberate, common enterprise pattern: modernize the hosting layer, keep the contract and data layers stable. You now understand not just the "how," but the "why," which is what will let you contribute with real judgment during your internship.

Final Outcomes Checklist

By completing this course, you should now be able to:

- ☒ Read any of the company's SOAP services from scratch, using the Module 9 checklist.
- ☒ Understand `[ServiceContract]` and `[OperationContract]` and why the company uses only these two WCF pieces.
- ☒ Understand how SoapCore turns those contracts into working SOAP endpoints, with no `ServiceHost`/`.svc`/IIS hosting involved.
- ☒ Read and build XML requests/responses confidently with `XmlDocument`.
- ☒ Trace a request end-to-end: Web Forms → SoapCore → Service → SQL Server → XML response → Web Forms.
- ☒ Debug a SOAP request locally and via raw captured traffic.
- ☒ Explain, in your own words, why the company's architecture looks the way it does.

Suggested next step: pick one real, currently-open ticket or a small existing service, and apply the

Module 9 reading checklist and Module 11 full-trace method to it directly — that's the fastest way to turn this course into real productivity.